GaitPPT: Parallel Part-based Transformer for Gait Recognition from Lidar Point Clouds

Balázs M. Bódis[†,✉]
bodis.balazs.mark@sztaki.hun-ren.hu
Gergő Nagy[†]
nagy.gergo.1@hallgato.ppke.hu
Csaba Benedek
benedek.csaba@sztaki.hun-ren.hu

[1] HUN-REN Institute for Computer Science and Control
Budapest, Hungary

[2] Pázmány Péter Catholic University
Faculty of Information Technology and Bionics
Budapest, Hungary

Abstract

Recognizing a person by gait, i.e., the way they walk, is a well-studied biometric task for which Lidar point clouds offer substantially greater robustness than 2D images: the sensor is less affected by lighting and captures the 3D shape of the body more directly, which most notably improves robustness to viewpoint changes. Recent methods for Lidar gait recognition have focused on adapting point-cloud or voxel encoders to the gait domain, but the way information is aggregated within and across frames has received less attention. Most systems take the maximum activation across frames, ignoring that different feature channels may peak at different times in the gait cycle. We introduce *GaitPPT*, a Parallel Part-based Transformer that targets two underexplored design axes: *where* attention is allocated within a frame, and *how* information is aggregated across frames. Spatially, GaitPPT runs multiple transformer encoder branches in parallel, each restricted to a horizontal slice of the body at a different vertical resolution; e.g., leg-height tokens cannot attend to torso-height tokens. Temporally, a lightweight learned module assigns every feature channel its own per-frame weighting, replacing the max-over-frames pooling used by prior work. On the SUSTech1K benchmark, GaitPPT-Base reaches 94.23% Cross-view Rank-1 with only 2.87 M parameters, an absolute improvement of 1.53% over the previous best point-based method, LidarGait++ (92.70% at 4.32 M), while using roughly one-third fewer parameters. These results suggest that an explicit vertical-structure prior is a useful inductive bias for Lidar-based gait recognition, and that frame-wise max pooling underutilizes the temporal structure of gait.

1 Introduction

Gait is a behavioral biometric that, unlike face or iris, can be acquired at distance and without the subject's explicit consent or cooperation. It has been studied for decades on silhouettes from RGB images [3, 4, 5, 11], but silhouettes are inherently in 2D, illumination-dependent and view-sensitive. Lidar overcomes these problems [17]: it captures 3D points, it is robust to lighting conditions and deployment is becoming increasingly affordable as the sensor

© 2026. The copyright of this document resides with its authors.
It may be distributed unchanged freely in print or electronic forms.
† Equal contribution / co-first authors. ✉ Corresponding author.

technology evolves. The release of the SUSTech1K [17] benchmark, with 25,239 sequences collected from 1,050 individuals under varied probe conditions (normal, bag, clothing, carrying, umbrella, uniform, occlusion, night), has triggered a focused line of work on point cloud-based gait recognition.

The first treatments of Lidar-based gait detection using deep-learning followed two design patterns, inherited from image-based recognition. The **first group** of methods projects the 3D points in each frame to a 2D depth image and applies a 2D CNN. The **second group** of methods voxelizes each point cloud into a 3D occupancy grid and applies a 3D CNN [23]. LidarGait [17] belongs to the first group. It adopts the GaitBase 2D-CNN backbone [5] on depth images to detect silhouette-based gait. The cost of this approach is the collapsing of one spatial dimension at the input. GaitCloud [23] belongs to the second group, aggregating the voxelized frames into a spatio-temporal tensor, processed by 3D convolutions. LidarGait++ [18] departs from both groups. It processes the point clouds directly via a four-stage PointNet++ Set Abstraction [16] network. It uses a Pyramid Point Pooling (P^3) [18] head, and introduces a size-aware learning mechanism that exploits absolute body height rather than normalized coordinates.

Two design choices repeated across these methods motivate our network architecture. **First**, local 3D structure is captured implicitly by stacking direction-dependent neighborhood operators. However, it has been shown that the discriminative gait signal is largely encoded along the vertical axis [3, 4, 14]: legs, torso, and head are stacked vertically, and their relative motion defines the gait cycle. LidarGait++ [18] partially engages this axis through its P^3 head, which partitions encoded features into height bins along the z-axis at multiple resolution scales (from the full height down to bins of scale $1/16$). This downstream readout is the only stage that explicitly reasons about height, and it acts on features produced by a direction-agnostic four-stage Set Abstraction encoder [16]; as a result, verticality engages only through the final pooling and not the representations that feed it. We push altitude structure prior upstream, embedding it directly in the spatial-reasoning operator. Our Parallel Part Transformer (PPT) constrains attention itself by masking multiple height resolution transformer encoders in parallel using different scales.

Second, among methods that aggregate per-frame features into a sequence-level descriptor, the consistent choice is a max-pool over time, inherited from the OpenGait framework [5, 6] and has been used by all recent Lidar gait methods that operate on a sequence of frame-level features [17, 18]. GaitCloud [23] forgoes this by collapsing the sequence into a single voxel-summed representation at the input, fusing temporal information before any feature extraction. We replace the temporal max-pool with a weighted temporal pooling module whose softmax weights are learned for each frame and channel combination, letting each channel combine information across the full sequence rather than relying on a single peak activation. With the above described design choices we introduce *GaitPPT*, a family of Parallel Part-based Transformers for Lidar gait recognition.

Our contributions are as follows:

(i) *GaitPPT*, a compact Lidar gait architecture built from three components: a *Point Tokenizer*, a shallow stack of Set Abstraction blocks that reduces each input point cloud to a compact set of anchor tokens; a *Parallel Part Transformer* that encodes the vertical-structure prior directly within attention, using one encoder branch per spatial-resolution scale rather than partitioning a single shared feature map, where attention is restricted to tokens sharing a vertical bin and a per-bin pooling yields the part tokens at each scale; and a *Weighted Temporal Pooling* module that replaces the frame-wise max-pooling that is the de facto standard in Lidar-based gait recognition, in which a lightweight MLP (Multilayer Percep-

tron) produces an attention logit for each frame and channel pair, softmax is applied across time independently per channel, and the output is a per-channel weighted sum. The number of parallel branches allows granularity when setting up the model for different tasks. Our base model, GaitPPT-Base, sets a new state of the art with Rank-1 accuracies of 94.23% on SUSTech1K [17] and 83.80% on FreeGait [10], surpassing the previous best, LidarGait++ [18], at 92.70% and 82.00% respectively.
(ii) We introduce a new training augmentation, namely: *axis-decoupled scaling* which draws the horizontal and vertical scaling factors independently, expanding coverage in body shape while staying close to the original training sets distribution; it improves overall accuracy, with its most notable gain on the clothing condition, the hardest probe setting.

2 Related Work

Lidar-based gait recognition has emerged as a distinct modality in deep-learning-based gait recognition [19]. Unlike camera silhouettes or depth-image projections, raw 3D point clouds preserve absolute scale and metric geometry while remaining robust to illumination and viewpoint, motivating a dedicated line of work. This shift was catalyzed by multi-modal Lidar benchmarks such as SUSTech1K [17] and FreeGait [10], which established standardized protocols for the task. This section organizes existing point-cloud approaches and positions our method among them.

2.1 Point Cloud Based Gait Recognition

Projection-based. LidarGait [17], introduced as the baseline method on SUSTech1K, treats Lidar-based gait recognition as a 2D problem after projection: each frame is rendered as a range-view depth image and processed by a GaitBase backbone [5], with frame embeddings aggregated by Set Pooling. This imports the engineering of silhouette gait pipelines at the price of discarding one spatial dimension at the input. On SUSTech1K, LidarGait remains strong under normal and occlusion conditions but degrades sharply on Umbrella, where missing returns most severely distort its range-view projection.
Voxel-based. GaitCloud [23] argues that depth projection collapses one spatial dimension and that point-wise networks ignore temporal continuity. It voxelizes each frame into a 3D occupancy grid, stacks frames into a 4D spatio-temporal tensor, and processes the result with a 3D residual encoder built around a ResNet9 backbone followed by a Horizontal Convolutional Pooling head. GaitCloud reaches strong cross-view accuracy but does so with a heavy 3D-convolution stack and a long training schedule.
Direct-point. LidarGait++ [18] operates on raw point coordinates and is the closest reference to ours. The encoder stacks PointNet++'s [16] Set Abstraction layers followed by a Pyramid Point Pooling (P^3) module that produces size-aware altitude bins and a size-aware encoding layer that injects relative altitudes into each Set Abstraction stage so the network does not collapse subjects of different statures.

2.2 Transformers on Point Clouds

Self-attention is a permutation-equivariant operator over set-structured inputs [20]. Point Transformer [24] and PCT [9] adopt attention as a general-purpose backbone for static point cloud tasks; P4Transformer [7] extends this to 4D action recognition by jointly attending

over space and time. These networks use attention as the dominant feature extractor: many stacked layers operating over thousands of input points. Using a transformer as a spatial feature extractor comes at a tradeoff: unlike convolutions, which efficiently exploit local geometric priors with a minimal memory footprint, the standard self-attention mechanism scales quadratically with the number of input tokens. For point clouds, this global point-to-point comparison leads to a prohibitive explosion in both computational overhead and VRAM consumption.

2.3 Part-based Methods

The standard practice in gait recognition is to partition the human body into distinct spatial regions, a gold standard established to capture the unique, identity-discriminative motion patterns of different anatomical parts [14]. This principle of spatial partitioning forms the core of highly successful silhouette-based methods like GaitSet [3] and GaitPart [4], which rely on horizontal pyramid pooling across fixed 2D stripes. LidarGait++ [18] extends this methodology to three-dimensional point clouds with P^3, partitioning the point cloud through size-aware spatial binning at multiple resolutions to produce a multi-scale representation. However, this approach partitions a single, shared backbone feature map instead of producing dedicated feature maps per resolution scale.

2.4 Temporal Aggregation

Temporal aggregation in silhouette gait is typically handled by set pooling [3] or 3D convolutions [11], while learnable attention pooling over sequences has a long lineage in additive attention [2] and NetVLAD-style aggregators [1]. In contrast, Lidar-based gait recognition relies almost exclusively on frame-wise max-pooling as the de facto standard for temporal aggregation [17, 18, 19]. This creates a significant structural limitation. While max-pooling operates independently per channel, it performs a hard, unweighted selection that discards the vast majority of the sequence's temporal context. Unlike attention, which applies a learned scalar weight per timestep, or 3D convolutions, which mix features across a fixed temporal window, max-pooling delegates the weighting of temporal information to the feature extractor backbone. Consequently, current Lidar methods struggle to capture the fine-grained, per-channel temporal dynamics necessary for robust gait representation, highlighting a critical bottleneck in how temporal sequences are currently processed.

3 Method

Addressing the limitations outlined in previous sections, we propose *GaitPPT*, an architecture designed specifically for gait recognition, summarized in Figure 1(a). First, a lightweight *Point Tokenizer* (Section 3.1), built from a stack of *Set Abstraction* (SA) blocks [16], produces per-frame tokens, as shown in Figure 1(b). A *Parallel Part Transformer* (Section 3.2) then processes these tokens with several encoders in parallel, each restricted to a different spatial resolution scale, yielding a multi-scale part representation, shown in Figure 1(c). Finally, a *Weighted Temporal Pooling* module (Section 3.3) aggregates the tokens along the time axis, learning independently for each part which frames are most informative (see Figure 1(d)).

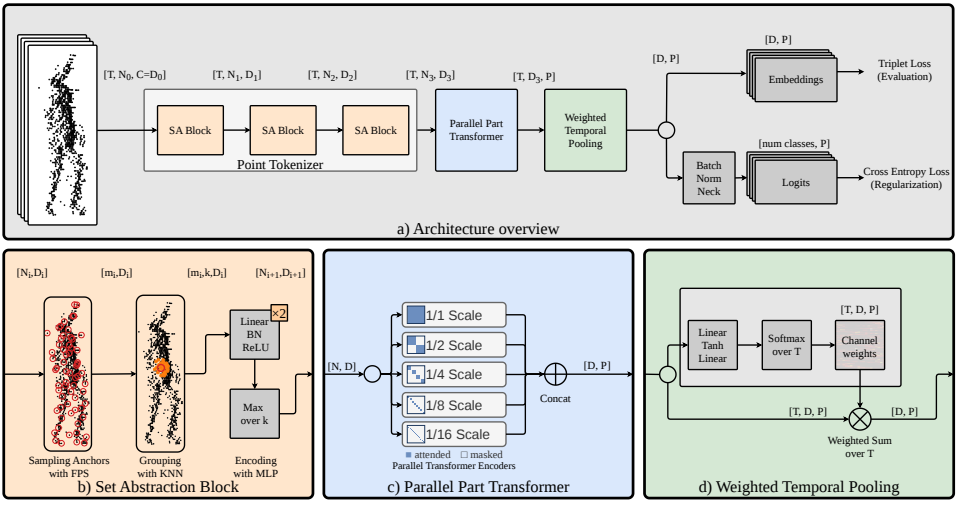


Figure 1: GaitPPT architecture. **(a)** Overall pipeline **(b)** Set Abstraction block. **(c)** Parallel Part Transformer with height mask. **(d)** Weighted temporal pool.

Each input sequence is represented as a tensor $\mathbf{X} \in \mathbb{R}^{T\times N\times C}$, where T is the number of frames per sequence, N is the per-frame point count, and C is the per-point input dimension (the three normalized spatial coordinates plus the original height coordinate for size-aware learning [18]). The network produces a part-wise embedding of dimension $D \times P$, where $D = 256$ is the token feature dimension and P is the number of parts, determined by the active scales. The resulting part embeddings are normalized by separate batch-norm necks and supervised with the combined triplet and cross-entropy loss of OpenGait [5, 6].

3.1 Point tokenizer

Applying a transformer directly on the point clouds is not trivial for gait recognition: self-attention scales quadratically with the number of tokens. We therefore precede the transformer with a tokenizer that distills each frame into a small, fixed set of expressive anchor tokens. LidarGait++ [18] showed that SA blocks [16] extract local spatial structure from raw point clouds effectively for this task, which makes them a natural choice for this tokenizer.

Each frame of 1024 points is reduced to $M = 128$ anchor tokens of dimension $D = 256$ by a cascade of three SA blocks with three steps: $512 \to 256 \to 128$, and $64 \to 128 \to 256$ respectively. At each stage, Farthest-Point Sampling (FPS) selects the anchor centres, k-Nearest Neighbors (k-NN) grouping forms the local neighborhoods, and displacement normalization makes the local geometry scale-invariant. The size-aware feature (original z coordinate adapted from LidarGait++ [18]) of each point plays two roles in the pipeline. First it is fed to the SA blocks during tokenization. Then, the post-FPS original z coordinate of every selected anchor is passed alongside the learned token. Later, it is given to the transformer of Section 3.2 as an anchor-level signal that drives the height-aware attention masking and scale-pooling bins. The tokenizer is deliberately compact, shortened to three SA blocks versus the four-stage encoder of LidarGait++, whose final stage is used to refine features at fixed channel width rather than increasing dimensionality. We omit this refinement stage

since spatial reasoning is performed by the downstream transformer.

3.2 Parallel Part Transformer

To encode the subject's spatial features across multiple scales, from local limb details to the overall body shape, the Parallel Part Transformer (PPT) architecture is designed to learn representations across multiple spatial resolution scales, acting as a *focused spatial reasoning module* rather than a feature extractor. Let $\mathcal{S}$ denote a set of *scales*, where a scale $1/s$ partitions the space into s vertical bins; for example, $\mathcal{S} = \{1/16, 1/1\}$ yields sixteen fine bins and one global bin. For each scale $1/s \in \mathcal{S}$, the module instantiates an independent transformer encoder branch with four attention heads, a GELU feed-forward block of width $4\times$ token dimension, and pre-LayerNorm normalization.

Adapting the size-aware spatial binning introduced by Shen *et al.* [18], bin assignment is computed per anchor directly from its original z-coordinate. By relying on the original coordinates rather than a per-sequence normalization, the bins remain tied to absolute spatial dimensions rather than subject-relative heights. An attention mask then blocks any pair of tokens whose anchors fall into different bins at scale $1/s$, ensuring that each layer attends only within a specific vertical strip. After the encoder, a per-bin mean+max pool over tokens produces s part vectors at that scale. The parts from all branches are concatenated, giving $P = \sum_{1/s \in \mathcal{S}} s$ parts in total (e.g., $16+1 = 17$ for the set above).

There are three notable observations about the design of the PPT layer. First, the height prior is expressed explicitly through the attention mask rather than discovered implicitly through depth. Second, scales are separated on independent branches that do not share weights but do process the same token sequence; specializing the granularity of the given binning using an attention mask, which restricts visibility. Tokens only attend to other points within their assigned bin. This allows the finer bins to concentrate on local limb structure to capture subtle spatial clues, while the global branch focuses on overall body shape, and larger, relative positions between body parts. Third, the parallel layout keeps the network shallow, with each branch implemented as a single encoder layer and no sequential refinement across scales.

3.3 Weighted temporal pooling

Previous gait recognition methods typically reduce the temporal dimension (T) using a max-pool, an order-invariant operation that retains only the maximum activation per channel. Our proposed module replaces this with a *Weighted Temporal Pool*, as shown in Figure 1(d). Softmax weights are computed for every (frame, channel) pair, enabling each channel to adaptively learn its own temporal weighting. This operation is applied after spatial processing, where each sequence is represented as a tensor of shape $T \times D \times P$, with each of the P parts processed independently. Let $x \in \mathbb{R}^{T\times D}$ denote the per-part feature sequence for a single part, with $x_{t,d}$ its entry at frame t and channel d. A two-layer MLP with weights $W_1, W_2 \in \mathbb{R}^{D\times D}$ produces a score tensor

$$s \;=\; \tanh(xW_1)\,W_2 \;\in\; \mathbb{R}^{T\times D}, \tag{1}$$

where $s_{t,d}$ is the score assigned to channel d at frame t. Softmax is applied on the logits along T, the temporal dimension, yielding the pooled per-channel descriptor. This is done

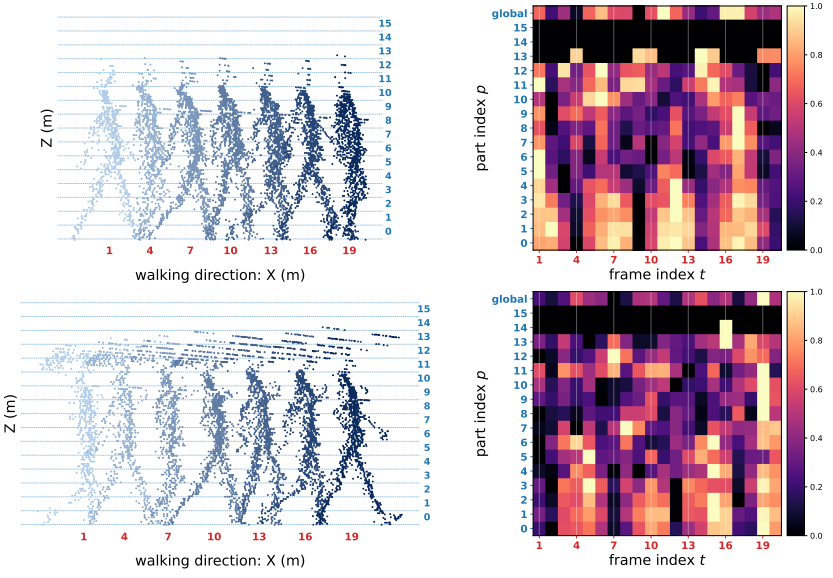


Figure 2: Weighted temporal-pool attention on two sequences of SUSTech1K. Top: normal walking; Bottom: with umbrella. *Left:* side-view Lidar over a 20-frame window, with 1/16 scale height-bin boundaries dashed and indexed. *Right:* per-(part, frame) attention, computed as the channel-averaged softmax with each row min-max normalized to $[0,1]$ for visibility.

independently for each channel:

$$\hat{x}_d = \sum_{t=1}^{T} \mathrm{softmax}_t(s_{:,d})_t \cdot x_{t,d}, \qquad d \in \{1,\dots,D\}, \tag{2}$$

The feature channel's weight is determined based on the extracted features: channels that respond to limb motion can sharpen onto the swing phase, while channels that encode quasi-static body shape can integrate over the full window. Figure 2 shows how these per-channel patterns aggregate at the part-token level for two SUSTech1K sequences: parts associated with the legs exhibit sharper, more concentrated temporal weighting than parts associated with the torso, reflecting the mixture of sharp and diffuse channels each part summarizes. Per-part attention (averaged across channels and normalized for better visibility) shows pronounced peaks in the leg bins and a more uniform profile in the global bin, and the peaks align with gait-cycle landmarks visible in the left-hand point clouds. In particular, the leg-cross moment of the swing phase — where the two legs collapse onto each other along the walking direction — appears at frame 4 of the top (normal) sequence and at frame 7 of the bottom (umbrella) sequence, and is recognizable as a distinctive activation column in the corresponding heatmaps: low-altitude bins light up or fall silent according to whether the channel responds to the cross-over or de-emphasises it, which would be invisible to a max-pool over frames. The pooled output has the same shape, $D \times P$, that a max-pool would produce, and adds approximately $2D^2 = 131,072$ parameters at $D = 256$.

3.4 Model variants

Given the parallel branch design of PPT, any set of scales $\mathcal{S} = \{1/s_1, \ldots, 1/s_n\}$ can be assigned across the parallel encoders, each branch operating independently at its designated scale. By adding or removing branches at specific scales we can guide the model toward learning features at those specific resolutions. Our headline model, *GaitPPT-Base*, uses two scales $\mathcal{S} = \{1/1, 1/16\}$ and outperforms LidarGait++ on both SUSTech1K and FreeGait while offering the best balance in compute requirements and accuracy. *GaitPPT-Large* employs the full scale pyramid $\mathcal{S} = \{1/1, \ldots, 1/16\}$, increasing the computational cost; this yields only a marginal gain on SUSTech1K but improves FreeGait accuracy significantly, the best of any variant and published method. *GaitPPT-Tiny* uses a single-scale encoder at $\mathcal{S} = \{1/1\}$ that captures the whole body at a single scale. The Tiny variant outperforms the far larger LidarGait baseline on both SUSTech1K and FreeGait at a fraction of the inference cost. A detailed ablation in Section 4.3 examines the contribution of each scale branch and motivates the three configurations presented above.

4 Experiments

Framework and code. All experiments are conducted within the OpenGait framework [5, 6]. The implementation will be released upon acceptance.

Dataset. Evaluation is conducted primarily on SUSTech1K [17], a 1,050-subject gait benchmark with 25,239 sequences captured by a 128-beam Velodyne VLS128 Lidar sensor across 12 viewpoints and eight covariate conditions. The official partition shipped with the dataset's release is adopted: 250 identities (6,011 sequences) for training and 800 identities (19,228 sequences) for testing. We additionally evaluate on FreeGait [10], a free-trajectory gait dataset captured outdoors by a 128-beam OUSTER-1 Lidar sensor with a split of 500 identities (7,659 sequences) in the train set and 695 (4,262 sequences) in the test set. We use it to test generalization outside SUSTech1K's controlled-capture protocol.

Protocol and baselines. Rank-1 accuracy is reported on both SUSTech1K (Cross-view protocol [17]) and FreeGait, with a per-condition breakdown for SUSTech1K and mean average precision (mAP) for FreeGait. Both Rank-1 and mAP use Euclidean distance as the similarity metric for evaluation. For our experiments we employ a single NVIDIA A100. To validate the adopted evaluation protocol, we reproduced both LidarGait and LidarGait++ on SUSTech1K — the latter required an extra GPU — obtaining overall Rank-1 accuracies of 86.98% (86.77% published) and 92.94% (92.70% published), respectively. Reproduction of LidarGait++ on FreeGait yields a Rank-1 of 80.63% (82.00% published) and an mAP of 85.86% (87.20% published).

4.1 Training

We use the standard OpenGait setup with its TripletSampler. Loss is an equally weighted sum of a triplet loss (0.2 margin) and a cross-entropy loss with temperature 30. Each batch is shaped $(14 \times 4, 20, 1024)$: 14 identities, 4 sequences per identity, 20 consecutive frames per sequence, and 1024 points drawn per frame. Dropout of 0.1–0.15 is applied in each transformer feed-forward block and on the temporal-pool scores. Optimization uses AdamW [12] ($\mathrm{lr} = 3 \times 10^{-3}$, weight decay 10^{-1}) with a cosine schedule decaying to 10^{-5} over 60,000 iterations on SUSTech1K and 120,000 iterations on FreeGait, no warmup.

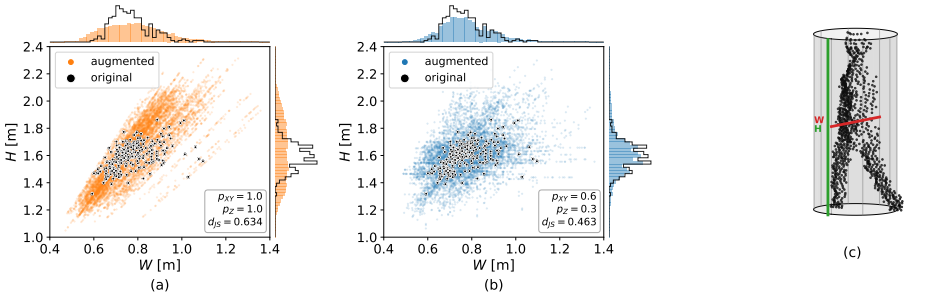


Figure 3: Effect of axis-decoupled scale augmentation on the SUSTech1K training population in body width/height (W, H) space. **(a)** LidarGait++ isotropic baseline ($p_{XY} = p_Z = 1$). **(b)** Our production recipe $(p_{XY}, p_Z) = (0.6, 0.3)$. **(c)** Geometric definition of W and H. Black dots: 250 train subjects; coloured dots: 40 Monte-Carlo augmentations per subject.

Augmentation is applied at training time only and comprises scale-aware normalization, two independent scale draws, and a horizontal flip. The XY plane is rescaled by $\mathcal{U}[0.8, 1.25]$ with probability of 0.6 on SUSTech1K and 0.1 on FreeGait, the Z axis by the same factor with probability 0.3 and 0.1, respectively. Approximately 28% of samples receive no scaling and preserve the original scale signal as a regularizing anchor. The asymmetry between the two probabilities encodes a physical prior: across the population, width (XY) varies substantially more than stature (Z), so the model benefits from greater exposure to XY perturbation, while Z is tightly distributed and heavier Z augmentation creates outliers in the training set. Figure 3 visualizes the resulting population coverage in body width $W = \max(X_{\text{ext}}, Y_{\text{ext}})$ and body height $H = Z_{\text{ext}}$ compared to the isotropic augmentation used by LidarGait++ [18]; our setting attains a smaller Jensen–Shannon distance d_{JS} to the original training distribution while covering body proportions that the baseline does not reach. By contrast, LidarGait++ applies a scale probability of 1.0 that shares an isotropic factor across all three axes simultaneously, never decoupled and never untouched; the per-condition difference between the two schemes is reported in Figure 6. The flip probability of 0.3 on SUSTech1K and 0.1 on FreeGait: horizontal flipping is the conventional augmentation for gait recognition as a regularization factor.

4.2 Main Results

Evaluation. We benchmark our model on SUSTech1K and FreeGait against recent state-of-the-art, Lidar-based methods, adopting their reported numbers from the original publications. For a fair comparison, the headline accuracies we report are single-run results from our original training seed, matching the single-run protocol used by these publications; we additionally report mean and standard deviation over five seeds in Section 4.3 as a stability analysis that prior work does not provide. On **SUSTech1K** (Tables 1 and 2), GaitPPT-Base attains 94.23% overall cross-view Rank-1 accuracy, surpassing the strongest prior method, LidarGait++ [18], by 1.53%. Despite its compact size, our Tiny variant remains highly competitive, matching the second-best published method, HMRNet [10], with only 1.05M parameters. GaitPPT-Large widens the margin further, improving over LidarGait++ by 1.62%. On **FreeGait** (Table 3), GaitPPT-Base surpasses the previous best by 1.80% in Rank-1 and 1.25% in mAP, while GaitPPT-Large extends this lead to 3.74% and 2.60%, respectively.

Input	Method	NM	BG	CL	CR	UB	UF	OC	NT	OA
Silhouette	GaitGL [GaitGL]	67.10	66.20	35.90	63.30	61.60	58.10	66.60	17.90	63.10
	GaitSet [8]	69.10	68.30	37.40	65.00	63.10	61.00	67.20	23.00	65.00
	GaitBase [8]	81.50	77.50	49.60	75.80	75.60	76.70	81.40	25.90	76.10
Depth Images	LidarGait [LidarGait]	91.80	88.64	74.56	89.03	67.50	80.86	94.53	90.41	86.77
	SimpleView [8]	72.30	68.80	57.20	63.30	49.20	62.50	79.70	66.50	64.80
Depth Images + Point Clouds	HMRNet [HMRNet]	92.70	92.30	79.60	90.30	83.10	86.20	95.20	90.40	90.20
Point Clouds	PointNet [PointNet]	35.30	31.00	22.30	23.10	14.80	23.80	36.00	26.10	25.30
	PointTransformer [PointTransformer]	53.20	48.10	32.00	43.20	39.10	41.80	47.90	47.10	44.40
	PointGait [PointGait]	68.60	65.20	48.10	56.80	35.60	55.00	68.80	61.70	57.60
	PointNet++ [PointNet++]	75.07	76.03	59.32	73.28	68.05	68.85	79.53	74.71	73.41
	PointTransformer V3 [PointTransformer V3]	81.42	81.21	63.15	77.57	72.42	76.23	84.91	79.77	78.11
	GaitCloud [GaitCloud]	89.82	90.30	79.17	89.65	85.79	89.02	94.39	90.62	89.20
	LidarGait++ [LidarGait++]	94.20	93.90	79.70	92.40	91.50	91.90	96.60	92.20	92.70
Point Clouds	GaitPPT-Tiny (ours)	94.11	91.44	76.01	89.59	87.79	90.89	94.82	89.19	90.24
	GaitPPT-Base (ours)	96.34	**95.64**	**81.50**	94.24	92.91	94.90	96.31	**92.49**	94.23
	GaitPPT-Large (ours)	**96.84**	95.59	81.08	**94.35**	**93.01**	**94.93**	**96.69**	92.35	**94.32**

Bold: best in column. gray :previous best.

Table 1: Per-condition Rank-1 accuracy on SUSTech1K. Conditions: NM (Normal), BG (Bag), CL (Clothing), CR (Carrying), UB (Umbrella), UF (Uniform), OC (Occlusion), NT (Night); OA is the Overall Rank-1 across all conditions, identical-view cases are always excluded.

Method	Input	# Param. (M)	GFLOPs	OAR1
LidarGait [LidarGait]	depth	8.03	3.44	86.77
HMRNet [HMRNet]	depth	–	–	90.20
LidarGait++ [LidarGait++]	PCs	4.32	1.02	92.70
GaitPPT-Tiny (ours)	PCs	1.05	0.92	90.24
GaitPPT-Base (ours)	PCs	2.87	1.13	94.23
GaitPPT-Large (ours)	PCs	6.15	1.74	**94.32**

Table 2: Method comparison on SUSTech1K [SUSTech1K]. # Param. (M) is the trainable parameter count in millions; GFLOPs is per-frame.

Method	Input	# Param. (M)	GFLOPs	OAR1	mAP
LidarGait [LidarGait, LidarGait++]	depth	–	–	74.20	80.70
HMRNet [HMRNet]	depth	–	–	80.80	86.50
LidarGait++ [LidarGait++]	PCs	7.02	1.79	82.00	87.20
GaitPPT-Tiny (ours)	PCs	1.24	1.14	78.54	84.30
GaitPPT-Base (ours)	PCs	4.09	1.35	83.80	88.45
GaitPPT-Large (ours)	PCs	8.26	1.96	**85.74**	**89.80**

Table 3: Method comparison on FreeGait [FreeGait]. # Param. (M) is the trainable parameter count in millions; GFLOPs is per-frame.

Model Complexity. On **SUSTech1K** (Table 2) all three GaitPPT variants perform on par or better in terms of accuracy and computational efficiency compared to other point based methods. *GaitPPT-Tiny* is the efficiency anchor: at 1.05M parameters and 0.92 GFLOPs per frame, it is the smallest and lowest-compute model in our comparison, nearly an 8× parameter reduction over LidarGait, while remaining competitive in accuracy. *GaitPPT-Base*, our recommended configuration, attains the better accuracy at a 34% smaller parameter footprint than LidarGait++, though at comparable per-frame compute. *GaitPPT-Large* adds capacity (6.15M parameters, 1.74 GFLOPs) that SUSTech1K accuracy does not reward. On **FreeGait** (Table 3), due to the difference between point density, slight architectural changes were made. Namely the k-NN grouping inside the SA blocks only selects 16 neighboring points for each anchor, compared to the 32 on SUSTech1K, to adapt the network to the sparser recordings in the benchmark. *GaitPPT-Large* performs better, indicating the extra intermediate scale branches start working when the data does not contain the finer details of the subjects. The BNNeck [BNNeck] (Fig. 1(a)) was changed, by increasing the output dimension, which forms the classification head for the cross-entropy regularization loss. This reflects the number of training identities; 250 on SUSTech1K and 500 on FreeGait. *GaitPPT-Base* outperforms it on both metrics while using roughly 42% fewer parameters (4.09M vs. 7.02M) and 25% fewer GFLOPs (1.35 vs. 1.79). Our smallest variant, *GaitPPT-Tiny*, requires only 1.24M parameters, about 5.7× fewer than LidarGait++, at just 1.14 GFLOPs, while still

surpassing the depth-based LidarGait. *GaitPPT-Large* reaches the best overall accuracy for only a modest increase in cost over LidarGait++.

4.3 Ablations

Variant	Scale Composition	# Param. (M)	OAR1
GaitPPT-Tiny	1/1	1.05	**89.63 ± 0.70**
	1/1 + 1/2	1.97	91.63
	1/16	2.02	93.34
GaitPPT-Base	1/1 + 1/16	2.87	**93.79 ± 0.63**
	1/1 + 1/2 + 1/4	3.02	92.24
	1/1 + 1/2 + 1/4 + 1/8	4.33	92.78
GaitPPT-Large	1/1 + 1/2 + 1/4 + 1/8 + 1/16	6.15	**94.29 ± 0.47**

Table 4: Parallel-branch scaling sweep of GaitPPT on SUSTech1K (OAR1). Intermediate steps between the named variants are a single run sweep.

Component progression	# Param. (M)	GFLOPs	R1
SA blocks [16]	0.13	0.72	73.64 ± 0.26
+ PPT branch (1/1 scale)	0.92	0.92	84.30 ± 2.78
+ PPT branch (1/16 scale)	2.74	1.12	73.18 ± 2.70
+ height-mask	2.74	1.12	91.89 ± 0.78
+ weighted temporal pool (= GaitPPT-Base)	2.87	1.13	93.79 ± 0.63

Table 5: Progressive ablation of GaitPPT-Base, results reported on SUSTech1K (Overall, Rank-1). Each row adds the specified component on top of the row above. # Param. (M) is the trainable parameter count in millions; GFLOPs is the per-forward-pass compute for a single input.

Our ablations establish the statistical stability of GaitPPT and validate the design decisions behind the architecture. All reported results are mean Rank-1 and sample standard deviation over five seeds on SUSTech1K: the original training seed used for the single-run results in Section 4.2, plus four additional seeds.

Branch Design. Table 4 shows the 3 GaitPPT variants: Tiny, Base, and Large. Additionally, as intermediate steps we report a single result for the remaining scale combinations. As discussed in Section 4.2, on the highly regulated SUSTech1K benchmark the large variant is not significantly more accurate. The results indicate that the most important scale is 1/16, which aligns with findings from LidarGait++ [18], where the P^3 module was shown to exploit the 1/16 scale with performance comparable to the full scale pyramid when tested individually.

Layer Contribution. Table 5 shows how each additional layer contributes to the feature extractor. All transitions are statistically significant under paired two-tailed t-tests ($p < 0.01$). A single full-body PPT branch reaches 84.30% Rank-1 accuracy. Adding a second branch at 1/16 scale without the explicit height prior decreases performance to 73.18%, showing that additional compute alone does not improve part separation. As illustrated in Figure 4, the part descriptors are formed by height pooling *after* the attention layer; consequently, unrestricted global self-attention over-smooths the features into near-duplicate part representations, collapsing the joint embedding space. This is reflected by an inter-part cosine

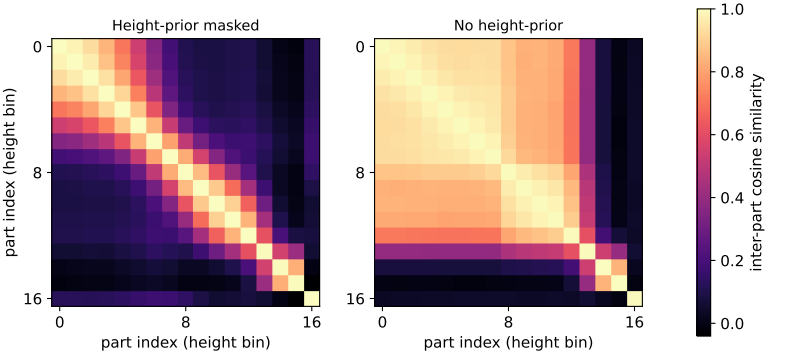


Figure 4: Inter-part cosine similarity of PPT.

similarity of 0.57 and an effective rank of only 6.5/17. Applying the height-prior mask confines each branch to its corresponding vertical band, encouraging specialization and reducing inter-part similarity to 0.28 while increasing the effective rank to 8.7/17. This raises Rank-1 accuracy to 91.89% without significant additional compute cost. The sparsest bins, 13–15, behave differently; excluding them, the masked and unmasked inter-part cosine similarities are 0.36 and 0.75, respectively, further confirming the over-smoothing effect. Finally, replacing standard temporal pooling with our weighted temporal pooling increases performance to 93.79%, completing GaitPPT. GaitPPT-Tiny separately explores removing height-based binning from the full final architecture.

Depth ablation. Using the Base single-layer model as the baseline (94.23%), increasing the depth to two and three transformer encoder layers yields accuracies of 94.06% and 92.52%, respectively. The corresponding model sizes increase from 2.87M parameters (1 layer) to 4.45M (2 layers) and 6.03M (3 layers).

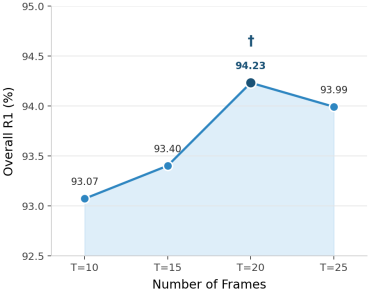


Figure 5: Ablation on sequence length T using GaitPPT-Base. Overall R1 (%) on SUSTech1K; † denotes the selected configuration.

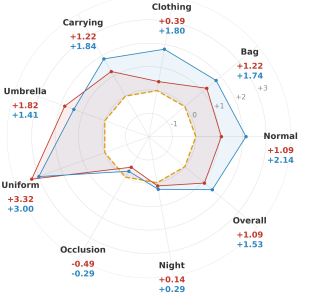


Figure 6: Per-condition R1 gain on SUSTech1K. Yellow: LidarGait++ baseline. Blue: GaitPPT-B with the default XY-Z scale augmentation. Red: the uniform-scale GaitPPT-B variant.

Sequence Length. The length of the input sequence has been a debated topic in gait recognition. Sequence length is a training-time hyperparameter for most state-of-the-art models, while the full sequence is processed at evaluation; GaitPPT follows this protocol. Figure 5

Method	Cross-domain R1 (%)	In-domain R1 (%)	Δ (%)
PointNet++ [16]	10.70	59.30	**-48.60**
LidarGait [17]	**23.00**	74.20	−51.20
LidarGait++ [18]	15.10	82.00	−66.90
GaitPPT-Tiny (ours)	17.22	78.54	−61.32
GaitPPT-Base (ours)	19.86	83.80	−63.94
GaitPPT-Large (ours)	18.73	**85.74**	−67.01

Table 6: Cross-dataset generalization, SUSTech1K → FreeGait. Cross-Domain R1 reports overall Rank-1 accuracy without finetuning. In-domain reports the results on the FreeGait benchmark. Δ reports the difference of the two.

shows Overall Rank-1 accuracy against training sequence length. GaitPPT-Base improves from 93.07% at 10 frames to 94.23% at 20 frames, but gains no further information beyond 20 frames. To verify that this behavior is architecture-dependent, we also reproduced LidarGait++ at 10, 15, and 20 training frames, obtaining 92.94%, 93.02%, and 93.08% Rank-1 accuracy, respectively, i.e., only a 0.2% spread within the margin of error. Note, that we retained its original 32×4 batch configuration (requiring three GPUs at 20 frames), whereas GaitPPT trains on a single GPU with a 14×4 batch. This asymmetry is expected: LidarGait++ relies on max-over-frames pooling, which cannot effectively exploit additional temporal context, while our weighted temporal pooling can. The saturation around 20 frames is also consistent with the findings of Zhang *et al.* [23] and the gait sequence analysis of Perry and Burnfield [14], as a gait cycle is rarely longer than 2 seconds.

Axis-Decoupled Scaling. Our scaling method, introduced in Section 4.1 has increased the overall Rank-1 accuracy of GaitPPT-Base, Figure 6 shows the per-condition changes. The largest improvement is observed in the clothing condition, where accuracy improves by +1.80% over LidarGait++, compared to only +0.39% for the uniform scaling variant. This suggests that separate horizontal and vertical scaling allows the model to better adapt to stature changes introduced by clothing. A notable improvement is also observed in the normal condition, where decoupled scaling nearly doubles the gain over LidarGait++ compared to uniform scaling (+2.14% vs. +1.09%).

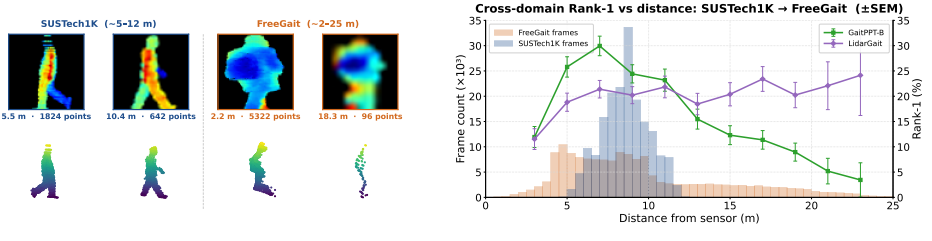


Figure 7: Cross-domain inputs and results.

Cross Domain Evaluation. Domain gap is also often discussed in gait recognition, and Table 6 reports the corresponding cross-domain performance. Projection-based methods, such as LidarGait, retain an advantage under domain shift because projecting the point cloud into a depth image suppresses much of the structural variation between datasets, although differences in point density and sparsity remain. This is also visible in Fig. 7: LidarGait operates

on projected depth images, whereas GaitPPT process the raw point cloud directly. Despite this disadvantage, GaitPPT-Base achieves a solid +4.76% improvement over LidarGait++, narrowing the projection-to-point performance gap from 7.9% to 3.1%. The trend within our own model family further suggests that representation, rather than model capacity, is the dominant factor, as GaitPPT-Large transfers slightly worse than Base despite its higher in-domain accuracy. GaitPPT also exhibits lower SEM at larger distances, as shown in Fig. 7. Nevertheless, zero-shot long-range recognition remains challenging for direct point-cloud methods. The dominant failure mode for both representations is the difference in view-angle diversity: SUSTech1K restricts subjects to approximately 45° movements along fixed paths, whereas FreeGait contains substantially less constrained, free-roaming trajectories.

5 Conclusion

We presented *GaitPPT*, a compact transformer architecture for Lidar-based gait recognition that adds two components to a shallow SA block-based point encoder: a set of *parallel* transformer branches whose attention is *masked by altitude scale*, and a *Weighted Temporal Pool* that replaces the max-over-frames aggregation of prior work. The height-masked attention embeds the vertical-structure prior directly in the spatial operator rather than recovering it as a downstream readout, while the weighted temporal pool restores per-channel access to the temporal variation that frame-wise max-pooling collapses. On both benchmarks GaitPPT surpasses the strongest published direct-point method, LidarGait++ [18] (92.70% on SUSTech1K, 82.00% on FreeGait), at substantially lower parameter cost: GaitPPT-Base reaches 94.23% Cross-view Rank-1 with 2.87 M parameters on SUSTech1K (+1.53%, 34% fewer parameters) and 83.80% with 4.09 M parameters on FreeGait (+1.80%, 42% fewer parameters). We also introduced an axis-decoupled scale augmentation, which draws the XY and Z scaling factors independently and thereby respects the asymmetric stature-versus-build distribution of human gait subjects, further improves robustness across covariate conditions, most notably clothing. Overall, these results support an explicit vertical-structure prior as an effective inductive bias for Lidar-based gait recognition, and identify the temporal-aggregation step as a productive axis for further investigation.

Ethics and Limitations. Gait is a biometric identifier that can be captured at a distance and without the subject's active cooperation; any system that increases recognition accuracy, including ours, carries a corresponding surveillance risk and should be deployed only under clear legal authorization and data-collection protocols that respect participant privacy. Training and evaluation are conducted solely on SUSTech1K [17] and FreeGait [10], whose subjects consented to collection for the stated research purpose.

Acknowledgment

This work was supported in part by the OTKA #143274 and TKP2021-NVA-01 projects of the Hungarian NRDI Office.

References

[1] Relja Arandjelović, Petr Gronat, Akihiko Torii, Tomáš Pajdla, and Josef Sivic. NetVLAD: CNN architecture for weakly supervised place recognition. In *Proc. CVPR*, pages 5297–5307, 2016.

[2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *Proc. ICLR*, 2015.

[3] Hanqing Chao, Yiwei He, Junping Zhang, and Jianfeng Feng. GaitSet: Regarding gait as a set for cross-view gait recognition. In *Proc. AAAI*, pages 8126–8133, 2019.

[4] Chao Fan, Yunjie Peng, Chunshui Cao, Xu Liu, Saihui Hou, Jiannan Chi, Yongzhen Huang, Qing Li, and Zhiqiang He. GaitPart: Temporal part-based model for gait recognition. In *Proc. CVPR*, pages 14225–14233, 2020.

[5] Chao Fan, Junhao Liang, Chuanfu Shen, Saihui Hou, Yongzhen Huang, and Shiqi Yu. OpenGait: Revisiting gait recognition towards better practicality. In *Proc. CVPR*, pages 9707–9716, 2023.

[6] Chao Fan, Saihui Hou, Junhao Liang, Chuanfu Shen, Jingzhe Ma, Dongyang Jin, Yongzhen Huang, and Shiqi Yu. Opengait: A comprehensive benchmark study for gait recognition towards better practicality. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pages 1–18, 2025.

[7] Hehe Fan, Yi Yang, and Mohan Kankanhalli. Point 4D transformer networks for spatio-temporal modeling in point cloud videos. In *Proc. CVPR*, pages 14204–14213, 2021.

[8] Ankit Goyal, Hei Law, Bowei Liu, Alejandro Newell, and Jia Deng. Revisiting point cloud shape classification with a simple and effective baseline. In *Proc. ICML*, pages 3809–3820, 2021.

[9] Meng-Hao Guo, Jun-Xiong Cai, Zheng-Ning Liu, Tai-Jiang Mu, Ralph R. Martin, and Shi-Min Hu. PCT: Point cloud transformer. *Computational Visual Media*, 7(2):187–199, 2021.

[10] Xiao Han, Yiming Ren, Peishan Cong, Yujing Sun, Jingya Wang, Lan Xu, and Yuexin Ma. Gait recognition in large-scale free environment via single LiDAR. In *Proc. ACM MM*, 2024.

[11] Beibei Lin, Shunli Zhang, and Xin Yu. Gait recognition via effective global-local feature representation and local temporal aggregation. In *Proc. ICCV*, pages 14648–14656, 2021.

[12] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *Proc. ICLR*, 2019.

[13] Hao Luo, Youzhi Gu, Xingyu Liao, Shenqi Lai, and Wei Jiang. Bag of tricks and a strong baseline for deep person re-identification. In *2019 IEEE/CVF (CVPRW)*, pages 1487–1495, 2019.

[14] Jacquelin Perry and Judith M. Burnfield. *Gait Analysis: Normal and Pathological Function*. SLACK Incorporated, Thorofare, NJ, 2 edition, 2010.

[15] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. PointNet: Deep learning on point sets for 3D classification and segmentation. In *Proc. CVPR*, pages 652–660, 2017.

[16] Charles R. Qi, Li Yi, Hao Su, and Leonidas J. Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. In *Proc. NeurIPS*, 2017.

[17] Chuanfu Shen, Chao Fan, Wei Wu, Rui Wang, George Q. Huang, and Shiqi Yu. LidarGait: Benchmarking 3D gait recognition with point clouds. In *Proc. CVPR*, pages 1054–1063, 2023.

[18] Chuanfu Shen, Rui Wang, Lixin Duan, and Shiqi Yu. LidarGait++: Learning local features and size awareness from LiDAR point clouds for 3D gait recognition. In *Proc. CVPR*, pages 6627–6636, 2025.

[19] Chuanfu Shen, Shiqi Yu, Jilong Wang, George Q. Huang, and Liang Wang. A comprehensive survey on deep gait recognition: Algorithms, datasets, and challenges. *IEEE Transactions on Biometrics, Behavior, and Identity Science*, 7(2):270–292, 2025.

[20] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proc. NeurIPS*, 2017.

[21] Rui Wang, Chuanfu Shen, Chao Fan, George Q. Huang, and Shiqi Yu. PointGait: Boosting end-to-end 3D gait recognition with point clouds via spatiotemporal modeling. In *Proc. IJCB*, pages 1–10, 2023.

[22] Xiaoyang Wu, Li Jiang, Peng-Shuai Wang, Zhijian Liu, Xihui Liu, Yu Qiao, Wanli Ouyang, Tong He, and Hengshuang Zhao. Point Transformer V3: Simpler, faster, stronger. In *Proc. CVPR*, pages 4840–4851, 2024.

[23] Shaoxiong Zhang, Hiromitsu Awano, and Takashi Sato. GaitCloud: Leveraging spatial-temporal information for LiDAR-base gait recognition with a true-3D gait representation. In *Proc. WACV*, pages 2849–2858, 2025.

[24] Hengshuang Zhao, Li Jiang, Jiaya Jia, Philip H. S. Torr, and Vladlen Koltun. Point transformer. In *Proc. ICCV*, pages 16259–16268, 2021.